

ЛР9 (3 пары / 6 часов)

«Статистика и отчёты: сколько задач в каждом статусе + мини-отчёт в консоль»

Что уже есть

- CRUD + JSON хранение + import/export/backup
- Единая валидация (TaskValidator)
- Поиск/фильтр/сортировка

Что делаем в ЛР9

Добавим "отчётность", чтобы приложение было полезнее:

1. Подсчитать **сколько задач** всего
 2. Сколько задач в статусах: **New / InProgress / Done**
 3. Показать мини-отчёт красиво и понятно
 4. (доп.) экспорт отчёта в текстовый файл
-

0) Итог ЛР9 (что должно работать)

✓ В меню есть пункт:

- 12) Статистика (отчёт)

✓ По нему выводится:

- всего задач
- New: N
- InProgress: N
- Done: N

✓ Если задач нет — пишет "Список пуст, статистика = 0".

✓ (доп. для 4–5) пункт:

- 13) Экспорт отчёта в файл `report_....txt`
-

1) Теория (важное, развернуто)

1.1. Зачем статистика

Это мини-аналитика:

- сколько осталось сделать
- сколько уже сделано
- как быстро движется работа

Даже в учебном проекте это хорошая практика: учимся считать и выводить результат.

1.2. Что такое “агрегация”

Агрегация — это “посчитать по группам”.

Например:

- посчитать задачи по статусам

В программировании это обычно:

- цикл `foreach`
- или `GroupBy` (но для новичков проще цикл)

1.3. Где хранить расчёты

Правило:

Core считает, **App** показывает.

Поэтому:

- метод статистики делаем в `TaskService`
 - меню только вызывает и печатает
-

2) Практика. Пара 1: создаём модель отчёта и метод в Core

Шаг 1. Создать папку Reports

В `TaskTracker.Core` :

1. Right click по проекту → Add → New Folder
2. Название: `Reports`

Шаг 2. Создать класс TaskStats.cs

1. Right click по `Reports` → Add → Class...
2. Название: `TaskStats.cs`
3. Add

Вставьте:

```
namespace TaskTracker.Core.Reports;

public class TaskStats
{
    public int Total { get; set; }
    public int NewCount { get; set; }
    public int InProgressCount { get; set; }
    public int DoneCount { get; set; }
}
```

Шаг 3. Добавить метод GetStats в TaskService

Откройте `TaskTracker.Core → Services → TaskService.cs`

Сверху добавьте using:

```
using TaskTracker.Core.Reports;
```

Внутри класса добавьте метод:

```
public TaskStats GetStats()
{
    var stats = new TaskStats();

    stats.Total = _tasks.Count;

    foreach (var t in _tasks)
    {
        if (t.Status == TaskStatus.New) stats.NewCount++;
        else if (t.Status == TaskStatus.InProgress) stats.InProgressCount++;
        else if (t.Status == TaskStatus.Done) stats.DoneCount++;
    }

    return stats;
}
```

Шаг 4. Сборка

Build → Build Solution → Build succeeded.

Шаг 5. Коммит №1

Сообщение:

- feat: add TaskStats report model and GetStats in TaskService

Commit All → Push.

3) Практика. Пара 2: добавляем пункт меню "Статистика" и красивый вывод

Шаг 6. Открыть Program.cs

TaskTracker.App → Program.cs

Шаг 7. Добавить пункт меню 12

В меню добавьте:

```
Console.WriteLine("12) Статистика (отчёт)");
```

Шаг 8. Реализовать обработку input == "12"

Добавьте блок:

```
if (input == "12")
{
    var stats = service.GetStats();

    Console.WriteLine();
    Console.WriteLine("Статистика задач");
    Console.WriteLine("-----");
    Console.WriteLine($"Всего задач: {stats.Total}");
    Console.WriteLine($"New:           {stats.NewCount}");
    Console.WriteLine($"InProgress:  {stats.InProgressCount}");
    Console.WriteLine($"Done:        {stats.DoneCount}");
    Console.WriteLine("-----");

    // простая проверка: сумма должна равняться total
    var sum = stats.NewCount + stats.InProgressCount + stats.
    DoneCount;
    if (sum != stats.Total)
```

```
        Console.WriteLine("ВНИМАНИЕ: сумма по статусам не рав  
на общему количеству!");  
  
        continue;  
    }  
}
```

Шаг 9. Проверка (F5)

Сценарий:

1. Добавьте 3 задачи
2. Поменяйте статусы так, чтобы была 1 New, 1 InProgress, 1 Done
3. Нажмите **12**
4. Должно вывести:
 - Total = 3
 - New = 1
 - InProgress = 1
 - Done = 1

Шаг 10. Коммит №2

Сообщение:

- **feat: add statistics command to console app**

Commit All → Push.

4) Практика. Пара 3: экспорт отчёта в файл (доп. но очень полезно) + отчёт ЛР9

Шаг 11. Добавить пункт меню 13 (экспорт отчёта)

В меню добавьте:

```
Console.WriteLine("13) Экспорт отчёта в файл");
```

Шаг 12. Реализовать обработку input == "13"

Нам нужна папка `reports` рядом с приложением.

Добавьте блок:

```
if (input == "13")
{
    try
    {
        var reportsFolder = Path.Combine(AppContext.BaseDirectory, "reports");
        Directory.CreateDirectory(reportsFolder);

        var stats = service.GetStats();

        var filePath = Path.Combine(
            reportsFolder,
            $"report_{DateTime.Now:yyyy-MM-dd_HH-mm-ss}.txt"
        );

        var lines = new List<string>
        {
            "TaskTracker Report",
            "-----",
            $"Date: {DateTime.Now:yyyy-MM-dd HH:mm:ss}",
            "",
            $"Total: {stats.Total}",
            $"New: {stats.NewCount}",
            $"InProgress: {stats.InProgressCount}",
            $"Done: {stats.DoneCount}",
        };
    }
}
```

```
        File.WriteAllLines(filePath, lines);

        Console.WriteLine("Отчёт сохранён: " + filePath);
    }
    catch (Exception ex)
    {
        Console.WriteLine("Ошибка экспорта отчёта: " + ex.Message);
    }

    continue;
}
```

Шаг 13. Проверка экспорта отчёта

1. Нажмите **13**
2. Должно написать путь файла
3. Откройте папку **reports** → там есть **report_...txt**
4. Откройте файл — там текст статистики

Шаг 14. Отчёт **docs/LR9_Report.md**

Создайте **docs/LR9_Report.md**:

```
# Отчёт ЛР9 – TaskTracker

## Что сделано
- Создан класс TaskStats (отчёт по статусам)
- В TaskService добавлен метод GetStats()
- В консольном приложении добавлен пункт:
  - 12) Статистика
- (доп.) Добавлен экспорт отчёта в файл:
  - 13) Экспорт отчёта в файл
```


Как проверить

- 1) Добавить 3 задачи
- 2) Поменять статусы так, чтобы были разные статусы
- 3) Нажать 12 и проверить числа
- 4) Нажать 13 и проверить файл report_*.txt в папке reports

Шаг 15. Коммит №3

Сообщение:

- feat: add report export and LR9 report

Commit All → Push.

5) Что сдавать (чек-лист)

✓ ссылка на GitHub

✓ docs/LR9_Report.md

✓ видео/скрин:

- статистика (12)
- экспорт отчёта (13) и открытый txt-файл

✓ минимум 3 коммита

6) Оценивание

"3" — работает статистика (12) и корректно считает

"4" — + экспорт отчёта в файл (13)

"5" — очень аккуратный вывод, проверка суммы, понятные сообщения об ошибках